DAA Mini Project REPORT

Banking System Simulator

Submitted in partial fulfillment of the requirements
for the award of the degree of

Bachelor of Technology
Computer Science & Engineering

Supervisor(s):
(Prof. (Dr.) Mohd. Izhar)

Submitted by:
Sarah Singh (35615602724)
Shruti Goyal (36815602724)
Tarun Kumar (90115602725)

Department of Computer Science & Engineering

Dr. Akhilesh Das Gupta Institute of Professional Studies

(Formerly ADGITM)

FC-26, SHASTRI PARK, NEW DELHI

Affiliated to

GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY

Sector - 16C Dwarka, Delhi - 110075, India

2024-28

DECLARATION

This is to certify that the material embodied in this mini Project titled “Banking System Simulator” being submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in CSE is based on our original work. It is further certified that this work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma. Our indebtedness to other works has been duly acknowledged at the relevant places.

(Sarah Singh)
Enrollment No. 35615602724

(Shruti Goyal)
Enrollment No. 36815602724

(Tarun Kumar)
Enrollment No. 90115602725

ACKNOWLEDGEMENT

We express our sincere gratitude to our supervisor, Prof. (Dr.) Mohd. Izhar, for his guidance, encouragement and valuable suggestions throughout the course of this mini project. His emphasis on analysing algorithms for correctness and complexity shaped the way this project was designed and evaluated.

We thank the Head of the Department of Computer Science & Engineering and the faculty members of Dr. Akhilesh Das Gupta Institute of Professional Studies for providing the academic environment and the facilities needed to complete this work. We are also grateful to our friends for testing the application and giving feedback during the demonstrations, and to our families for their constant support.

Sarah Singh
Shruti Goyal
Tarun Kumar

ABSTRACT

Banking software must keep every balance correct even when many requests arrive at the same time, when a client repeats a request after a timeout, and when a server fails half way through moving money. This report presents the design, analysis, implementation and evaluation of the Banking System Simulator, an educational full-stack web application that uses simulated money and is built with React, Node.js, Express and MongoDB.

The report formulates the core of the system as eight algorithms: exact amount parsing in integer paise, an atomic conditional balance update, idempotent request execution, a transfer with transactional or compensating atomicity, randomized account number generation, indexed and paginated history retrieval, hash-based aggregation for analytics, and password hashing with token authentication and rate limiting. For each algorithm the report gives pseudocode, time and space complexity, and an argument that it preserves four invariants: no overdraft, conservation of money, exactly-once effect and agreement between balances and the ledger. The system is documented with a context diagram, data flow diagrams of levels 0, 1 and 2, an entity-relationship diagram, a use case diagram and four detailed use cases.

The implementation passes 15 automated integration tests on a transactional and on a non-transactional MongoDB. Experiments show that fifty concurrent withdrawals of Rs. 100 against a balance of Rs. 1,000 were all accepted by a naive read-modify-write update in 20 of 20 trials, whereas the atomic update accepted exactly ten; that 300 concurrent transfers conserved the total money; and that a compound index kept the history query near one millisecond for collections of up to 100,000 documents, against 99 ms without it. The evaluation also uncovered and repaired a double-credit defect on standalone servers.

Keywords: *banking ledger, atomic update, idempotency, concurrency control, MongoDB, algorithm analysis, complexity.*

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION AND PROBLEM STATEMENT 1

1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Significance of the Problem 3
1.5 Scope and Limitations 3
1.6 Organization of the Report 4

CHAPTER 2: BACKGROUND STUDY AND RELATED CONCEPTS 5

2.1 Ledgers and the Double-Entry Principle 5
2.2 Concurrency and the Lost-Update Problem 5
2.3 Atomicity and Database Transactions 5
2.4 Idempotency of Requests 6
2.5 Authentication and Password Storage 6
2.6 Indexing, Pagination and Aggregation 6
2.7 Existing Approaches and the Gap 7

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN 8

3.1 System Architecture 8
3.2 Context Diagram 8
3.3 Data Flow Diagrams 9
3.4 Entity-Relationship Diagram 11
3.5 Use Case Diagram 12
3.6 Use Cases 13

CHAPTER 4: ALGORITHM DESIGN 16

4.1 Design Overview and Invariants 16
4.2 Algorithm 1: Exact Amount Parsing 16
4.3 Algorithm 2: Atomic Conditional Balance Update 17
4.4 Algorithm 3: Idempotent Execution 18
4.5 Algorithm 4: Transfer with Atomicity and Compensation 19
4.6 Algorithm 5: Unique Account Number Generation 20
4.7 Algorithm 6: Filtered, Sorted and Paginated History 21
4.8 Algorithm 7: Analytics Aggregation 22
4.9 Algorithm 8: Authentication and Rate Limiting 22
4.10 Complexity Summary 23
4.11 Correctness Arguments 24

CHAPTER 5: IMPLEMENTATION 25

5.1 Technology Stack 25
5.2 Backend Implementation 25
5.3 REST Interface 26
5.4 Frontend Implementation 27
5.5 Security Measures 28
5.6 Testing Strategy 28

5.7 Challenges Faced and Solutions 28
5.8 Combined Experiment: Queue, Stack and Searching 29
CHAPTER 6: RESULTS AND DISCUSSIONS 33
6.1 Experimental Setup 33
6.2 Functional Test Results 33
6.3 Experiment 1: Concurrent Withdrawals 33
6.4 Experiment 2: Idempotency under Duplicate Requests 34
6.5 Experiment 3: Conservation under Concurrent Transfers 35
6.6 Experiment 4: Operation Latency 35
6.7 Experiment 5: Throughput under Contention 36
6.8 Experiment 6: History Retrieval and the Index 37
6.9 Experiment 7: Analytics Aggregation 38
6.10 Experiment 8: Cost of Password Hashing 38
6.11 Discussion 39
CHAPTER 7: CONCLUSION AND FUTURE WORK 41
7.1 Conclusion 41
7.2 Future Work 41
REFERENCES 43

LIST OF FIGURES

Figure No. and Title **Page No.**

Figure 3.1: Three-tier architecture of the Banking System Simulator 8
Figure 3.2: Context diagram of the Banking System Simulator 9
Figure 3.3: Level 0 data flow diagram 10
Figure 3.4: Level 1 data flow diagram 10
Figure 3.5: Level 2 data flow diagram of process 3.0 (transaction processing) 11
Figure 3.6: Entity-relationship diagram 12
Figure 3.7: Use case diagram 13
Figure 4.1: Flowchart of the atomic conditional balance update 17
Figure 4.2: Flowchart of the transfer algorithm with idempotency and compensation 19
Figure 5.1: Dashboard with balance, statistics and charts 27
Figure 5.2: Transfer page with recipient verification 27
Figure 5.3: Transaction history with search, filters and pagination 27
Figure 5.4: Analytics page with charts 28
Figure 5.5: Deposits vs withdrawals per account (seaborn bar chart) 32
Figure 6.1: Accepted withdrawals and overdraft violations, naive vs atomic 34
Figure 6.2: Mean latency of ledger operations with 95th percentile whiskers 36
Figure 6.3: History page latency with and without the compound index 37
Figure 6.4: bcrypt hashing time against the cost factor 39

LIST OF TABLES

Table No. and Title **Page No.**

Table 1.1: Inputs, processes and outputs of the system 2
Table 2.1: Comparison of balance-update strategies 7
Table 3.1: Use case UC-01: Register Account 13
Table 3.2: Use case UC-02: Deposit Money 14
Table 3.3: Use case UC-03: Transfer Money 14
Table 3.4: Use case UC-04: View and Export Transaction History 14
Table 4.1: Time and space complexity of the algorithms 23
Table 5.1: Tools and libraries used 25
Table 5.2: REST API endpoints 26
Table 5.3: Challenges and their solutions 28
Table 5.4: Data structures used in the combined experiment 29
Table 5.5: Account search on 100,000 accounts (average of 1,000 lookups) 31
Table 6.1: Results of Experiment 1 (20 trials of 50 concurrent withdrawals) 34
Table 6.2: Results of Experiment 2 (20 trials, 10 concurrent duplicates each) 35
Table 6.3: Results of Experiment 3 35
Table 6.4: Results of Experiment 4: latency in milliseconds (200 sequential calls each) 36
Table 6.5: Results of Experiment 6: latency of the first history page (ms) 37
Table 6.6: Results of Experiment 6 (continued): latency by page number with the index (ms) 37
Table 6.7: Results of Experiment 8: bcrypt time per operation (ms) 38

CHAPTER 1: INTRODUCTION AND PROBLEM STATEMENT

1.1 Background

Banking is one of the most demanding application domains in software engineering. A bank must record every movement of money exactly once, must never let an account fall below zero because of a race between two requests, and must be able to explain every balance as the sum of the transactions that produced it. These requirements look simple on paper, but they become hard as soon as many customers use the system at the same time over an unreliable network. Two browser tabs may submit a withdrawal at the same instant, a mobile user may tap the Pay button twice because the screen froze, and a server may crash half way through moving money from one account to another. A correct banking system is therefore not just a set of screens connected to a database; it is a carefully designed collection of algorithms that preserve a small number of invariants under concurrency and failure.

The simulator is a full-stack application. The user interface is a single-page React application, the server is a Node.js and Express REST API, and the data is stored in MongoDB. A new customer registers, receives a unique 12-digit account number and a demonstration credit of Rs. 10,000, and can then deposit, withdraw and transfer money, search and export the transaction history, and view analytics. No real payment network or real bank account is connected to the system at any point.

1.2 Problem Statement

The problem addressed by this project is to design, analyse and implement the core algorithms of a simple retail banking ledger so that the following properties hold even when requests arrive concurrently, are repeated by the client, or fail part-way through:

- **No overdraft:** the balance of an account never becomes negative.
- **Conservation of money:** a transfer moves money from one account to another without creating or destroying any, so the total money in the system changes only through deposits and withdrawals.
- **Exactly-once effect:** a request that is retried with the same identity changes the system once, never twice.

- **Exact arithmetic:** amounts are represented without floating-point rounding errors.
- **Isolation of customers:** a customer can read and change only their own data.
- **Efficient retrieval:** the history and analytics of an account remain fast as the number of transactions grows.

The inputs, processes and expected outputs of the system are summarised in Table 1.1.

Table 1.1: Inputs, processes and outputs of the system

Inputs	Processes	Outputs
Name, e-mail and password for registration; credentials for login	Validation, bcrypt hashing, account number generation, JWT issuing and verification	Customer record, 12-digit account, Rs. 10,000 welcome credit, signed token
Deposit or withdrawal amount with an optional description and an idempotency key	Exact parsing to paise, atomic conditional balance update, ledger entry creation, duplicate detection	Updated balance, immutable ledger record with reference number, or a clear error
Recipient account number and amount	Recipient verification, debit and credit as one logical unit, compensation on failure	Two linked ledger records (sent and received) sharing one transfer group
Search text, type, date range, sort order, page number	Filtered, sorted, paginated index query; CSV generation with injection protection	Page of transactions, total count, downloadable CSV file
Date range and grouping (month or year)	Group-by aggregation over ledger records	Totals, net cash flow, monthly series, distribution by type

1.3 Objectives

- To design a three-tier architecture (React client, Express API, MongoDB database) for a simulated banking system and document it with a context diagram, data flow diagrams, an ER diagram and use case models.
- To formulate each core operation as an algorithm with pseudocode, a flowchart where useful, and an analysis of its time and space complexity.
- To prove, using invariants, that the ledger algorithms preserve the no-overdraft, conservation and exactly-once properties.
- To implement the system with industry-standard tools and to verify it with an automated test suite of 15 integration tests.
- To measure the behaviour of the algorithms experimentally, including a comparison of a naive read-modify-write balance update with the atomic

conditional update, the effect of an index on history retrieval, and the cost of bcrypt hashing.

1.4 Significance of the Problem

Errors in financial software are expensive and often invisible until money has already moved. The classic lost-update anomaly, in which two concurrent operations read the same balance and each write back a result computed from the stale value, is a well-known source of such errors [3], [14]. The retry problem is equally common: if a payment request times out, the client cannot know whether the server processed it, and a naive retry may charge the customer twice. Payment providers solve this with idempotency keys [8], and databases solve the concurrency problem with atomic operations and transactions [1], [2].

1.5 Scope and Limitations

The scope of the project is a single-currency (INR) savings-account simulator with deposit, withdrawal, transfer between customers, history with search and export, analytics, profile management and a light and dark theme. The following are outside the scope:

- Connection to any real bank, card network, UPI or payment gateway; all money is simulated.
- Multiple accounts per customer, interest calculation, loans, cards and bill payments.
- An administrator dashboard. A role field exists in the user model for future use, but no administrative screens were built.
- Server-side revocation of tokens at logout. The JSON Web Token is stateless, so logout discards the token in the browser only.
- Formal verification and large-scale load testing; the experiments in this report were run on a single development machine.

1.6 Organization of the Report

Chapter 2 reviews the underlying concepts. Chapter 3 presents the analysis and design models. Chapter 4 gives the eight main algorithms with complexity and correctness

arguments. Chapter 5 describes the implementation. Chapter 6 reports the experiments. Chapter 7 concludes and lists future work, followed by the references.

CHAPTER 2: BACKGROUND STUDY AND RELATED CONCEPTS

This chapter explains the ideas that the algorithms of Chapter 4 depend on.

2.1 Ledgers and the Double-Entry Principle

A ledger is an append-only record of events. Instead of storing only the current balance, an accounting system stores every deposit, withdrawal and transfer, so that the balance can be reproduced and audited. Double-entry bookkeeping records each transfer as a debit in one account and a credit in another, and the two entries sum to zero [13]. The simulator follows this idea: every change of balance creates a transaction document with the type, amount, balance before and after, a unique reference and a timestamp, and a transfer creates two linked documents (TRANSFER_SENT and TRANSFER_RECEIVED). Even the Rs. 10,000 welcome credit is stored as a genuine deposit, so that history, analytics and balance always agree.

2.2 Concurrency and the Lost-Update Problem

When several requests operate on the same account their reads and writes can interleave. With a balance of 1000 and two withdrawals of 600 arriving together, both requests may read 1000, both checks succeed, and both write 400. The customer has received 1200 from an account that held 1000. This anomaly is called a lost update [14] and cannot be fixed by checking the balance in the application, because the check and the update are separate steps.

The standard remedies are **pessimistic locking**, which locks the account before reading it but reduces concurrency and risks deadlock; **optimistic concurrency control**, which writes only if a version number is unchanged and retries otherwise [3]; and the **atomic conditional update** used in this project, which moves the check into the write: the database subtracts the amount only if the balance is at least the amount, as one indivisible operation. It is the database equivalent of the compare-and-swap instruction and needs neither a lock held by the application nor a retry loop.

2.3 Atomicity and Database Transactions

A transfer touches two accounts and two ledger documents. If the program fails after the debit but before the credit, money vanishes. The ACID properties of transactions

[2], [1] prevent this: either all steps take effect or none do. MongoDB supports multi-document transactions on replica sets [7], and MongoDB Atlas always runs as a replica set. A developer machine often runs a standalone server without transactions, so the simulator works in both settings: it uses a transaction when it can, and otherwise relies on atomic single-document updates plus a **compensating action** that reverses earlier changes if a later step fails. This pattern is known as a saga [9].

2.4 Idempotency of Requests

An operation is idempotent if performing it several times has the same effect as performing it once. “Add Rs. 500 to the balance” is not, and networks make such operations dangerous: if a response is lost, the client may send the request again. Payment providers solve this with an **idempotency key** attached to each logical request; the server stores the keys it has processed and returns the stored result for a repeated key [8]. The check “have I seen this key?” followed by the action is itself a check-then-act race, so the simulator adds a unique database index on the pair (user, key): whichever copy inserts its ledger record first wins, and the other receives a duplicate-key error that it converts into the stored result.

2.5 Authentication and Password Storage

The simulator uses **JSON Web Tokens** [5]: at login the server signs a token containing the user identifier and a one-day expiry with an HMAC key known only to the server, and the client sends it with every request. The token is stateless, which keeps the algorithm simple and fast but means that it cannot be revoked before it expires. Passwords are stored only as **bcrypt** hashes [4]. Bcrypt is deliberately slow, has a tunable cost (each increase of one doubles the work) and embeds a random salt, which defeats pre-computed tables. A cost of 12 is used; Section 6.10 measures what this means in milliseconds.

2.6 Indexing, Pagination and Aggregation

A history page shows ten records of a possibly huge collection, newest first. Without an index the database scans the whole collection, filters the user’s documents and sorts them. A **compound index** (B-tree) on the user and the creation time lets the database jump to the user’s first record and read in sorted order, so the cost depends on the page and not on the collection size [6]; this is a space-time trade-off, since the index uses

storage and slows inserts slightly. Offset pagination skips (page - 1) * k records, so deep pages cost more. Analytics such as totals per month are group-by queries evaluated in linear time with a hash table of group keys. Money-in is defined as deposits plus transfers received and money-out as withdrawals plus transfers sent; as each user stores only their own side of a transfer, no transfer is counted twice.

2.7 Existing Approaches and the Gap

Table 2.1 compares typical balance-update strategies. Many tutorial banking applications use the first row, which is unsafe under concurrency. The simulator combines the atomic conditional update with idempotency keys and a compensation fallback, so that it is correct on both transactional and non-transactional databases.

Table 2.1: Comparison of balance-update strategies

Strategy	Concurrency safe	Retry safe	Remarks
Read, check, write back (naive)	No	No	Lost updates and overdrafts under load
Pessimistic row locking	Yes	No	Reduces parallelism; deadlock risk
Optimistic version check with retry	Yes	No	Extra read and retry loop under contention
Atomic conditional update (this project)	Yes	With idempotency key	Single operation; no lock; no retry loop
Atomic update + transaction + compensation (this project)	Yes	Yes	Correct on replica sets and standalone servers

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

This chapter describes the system at several levels of abstraction. It begins with the overall architecture and then refines the view through the context diagram, the data flow diagrams of levels 0, 1 and 2, the entity-relationship diagram and the use case model.

3.1 System Architecture

The simulator follows a classic three-tier architecture, shown in Figure 3.1. The **presentation tier** is a React single-page application served by Vite during development. It keeps the JSON Web Token in the browser, attaches it to every request, and generates a fresh idempotency key for each money-moving form submission. The **application tier** is an Express server whose middleware chain applies security headers (Helmet), cross-origin rules (CORS), a ten-kilobyte body limit and a rate limiter on the login and registration routes, then authenticates the caller before the routes run. All money movement is delegated to a single ledger service, which is the only module allowed to change balances. The **data tier** is MongoDB, accessed through Mongoose, with three collections and the indexes listed in Section 3.4.

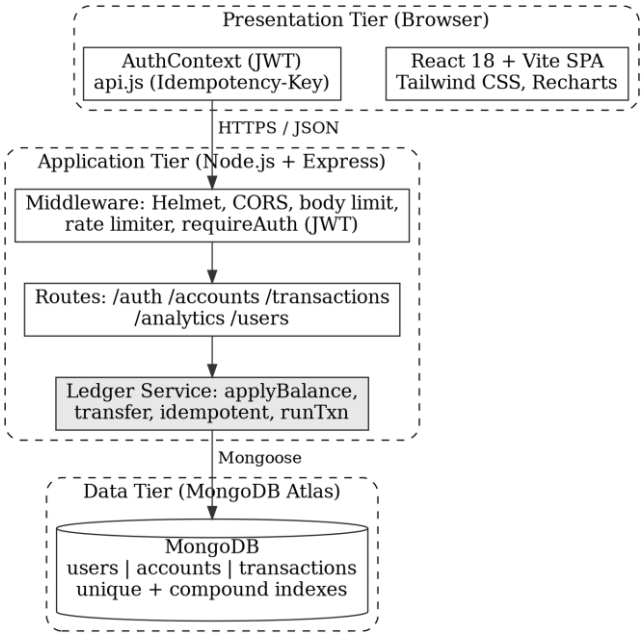


Figure 3.1: Three-tier architecture of the Banking System Simulator

3.2 Context Diagram

The context diagram in Figure 3.2 shows the whole simulator as one process and the external entities that exchange data with it. The **customer** supplies registration data,

credentials, money requests, history filters and profile updates, and receives tokens, balances, receipts, history pages, CSV files and analytics. The **database** receives read and write queries, including the atomic balance updates, and returns the stored records. A **developer or examiner** runs the automated test suite and the health check and receives pass or fail results; this entity is included because the test suite is a first-class part of the deliverable.

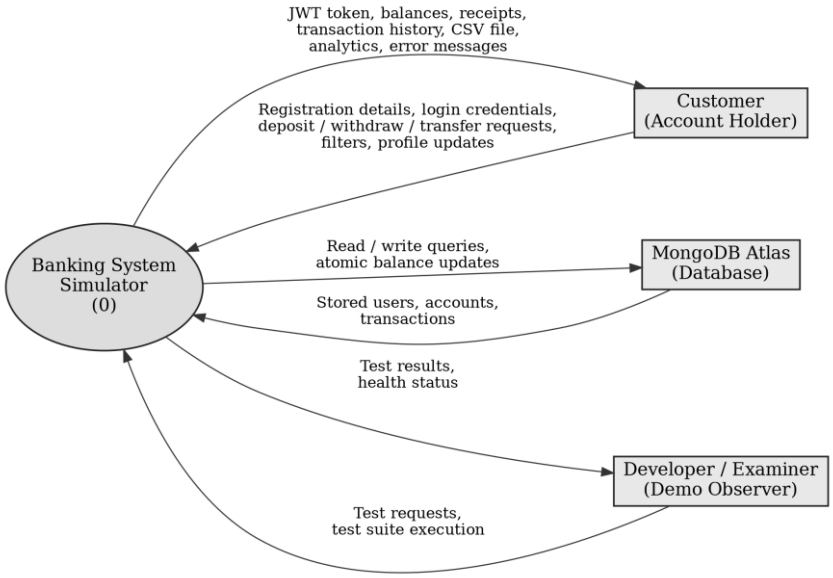


Figure 3.2: Context diagram of the Banking System Simulator

3.3 Data Flow Diagrams

3.3.1 Level 0 DFD

The level 0 diagram of Figure 3.3 keeps the simulator as a single process but exposes the three data stores that it reads and writes: D1 holds users, D2 holds accounts with their balances, and D3 holds the immutable transaction records. The customer is still the only external entity that initiates requests.

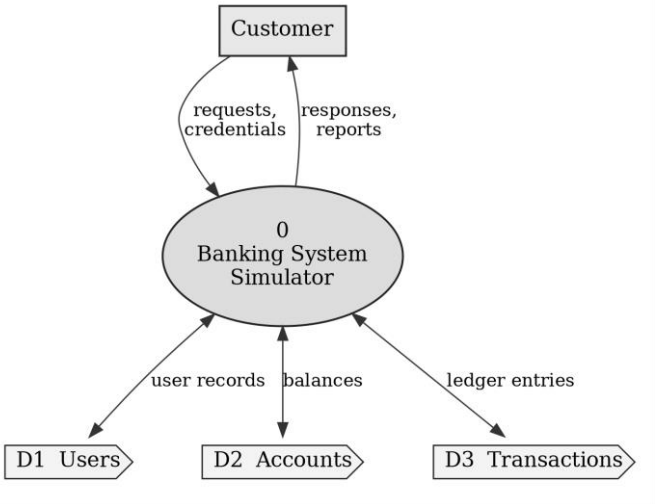


Figure 3.3: Level 0 data flow diagram

3.3.2 Level 1 DFD

Figure 3.4 decomposes the system into six processes. Process 1.0 handles registration and login and, during registration, triggers the welcome credit by calling process 3.0. Process 2.0 serves account details and recipient verification. Process 3.0 executes deposits, withdrawals and transfers and is the only process that writes to the balance field of D2. Process 4.0 retrieves, filters and exports history, process 5.0 computes analytics, and process 6.0 manages the profile. The processes 4.0 and 5.0 only read D3, so they cannot disturb the ledger.

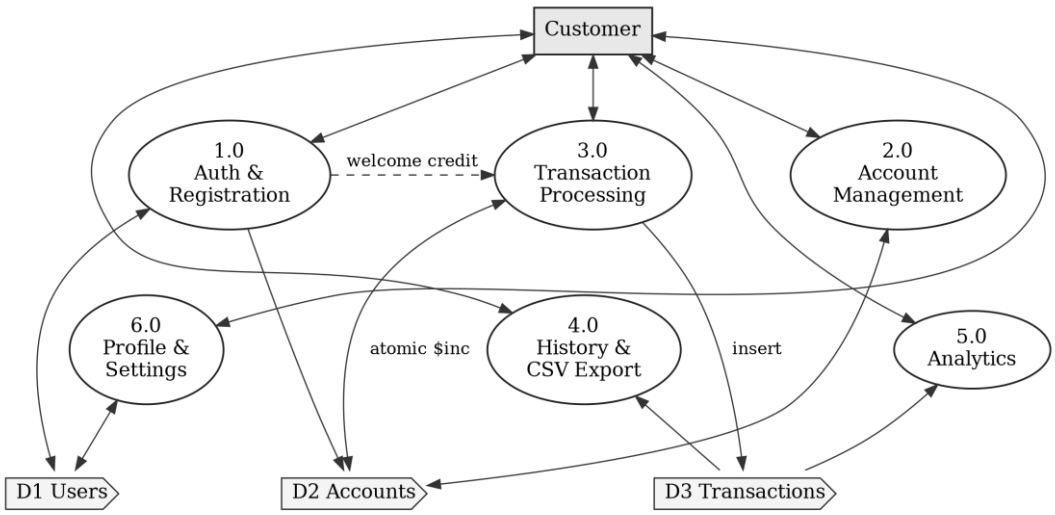


Figure 3.4: Level 1 data flow diagram

3.3.3 Level 2 DFD of Transaction Processing

The most important process is 3.0, expanded in Figure 3.5. The request is first validated (3.1), where Zod checks the shape of the body and the exact parser converts the amount

into integer paise. The idempotency check (3.2) looks up the pair of user and key in D3; if the pair is found, the stored result is returned to the customer and nothing else happens. Otherwise the atomic debit (3.3) tests and subtracts the amount in one operation on D2, the atomic credit (3.4) adds it to the recipient, and the ledger entries (3.5) are inserted into D3. If the debit finds insufficient funds or any later step fails, the compensation process (3.6) reverses the balance changes and returns an error to the customer.

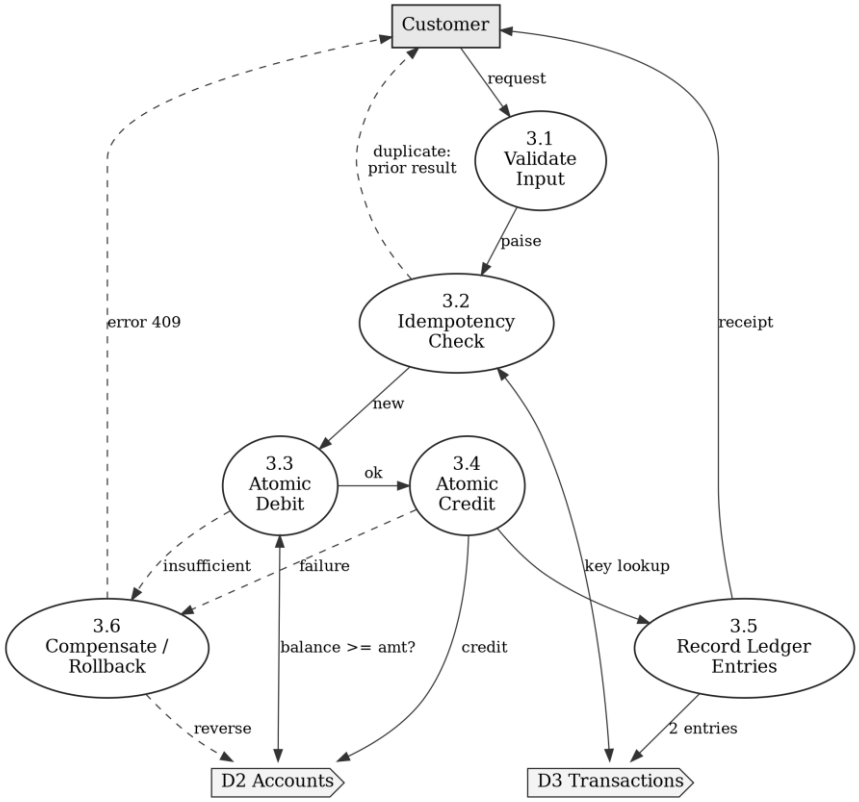


Figure 3.5: Level 2 data flow diagram of process 3.0 (transaction processing)

3.4 Entity-Relationship Diagram

The data model has three entities, shown in Figure 3.6. A **User** owns exactly one **Account**, and an Account has many **Transaction** records. The Transaction entity also stores the user identifier directly; this redundancy is intentional because it lets history and analytics queries filter by user using a single compound index without joining to the account.

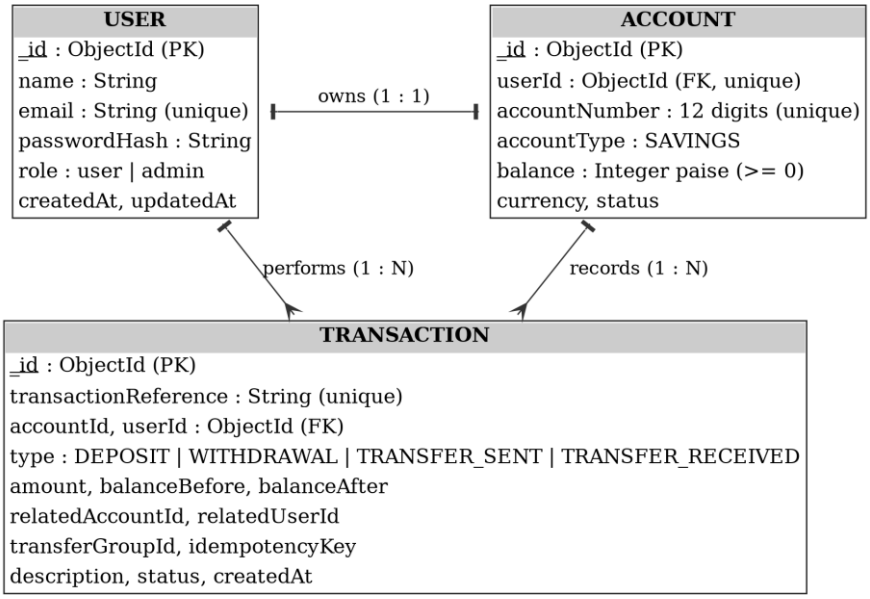


Figure 3.6: Entity-relationship diagram

The attributes and their types are listed in the diagram. The model enforces these constraints: the e-mail, the account number, the user identifier of an account and the transaction reference are unique, the balance is an integer of at least zero, and an amount is at least one paisa.

Two indexes on the Transaction collection carry the algorithmic load: the compound index on (userId ascending, createdAt descending) that serves history, export and analytics, and the unique partial index on (userId, idempotencyKey) that serves duplicate detection.

3.5 Use Case Diagram

Figure 3.7 shows the functions that the system offers. The customer is the primary actor and interacts with thirteen use cases. Transfer Money includes Verify Recipient, because the interface always resolves the account number to a holder name before the money is sent, and Export CSV extends View Transaction History because the export reuses the filters of the history page. MongoDB Atlas is shown as a secondary actor because the main use cases read from or write to it.

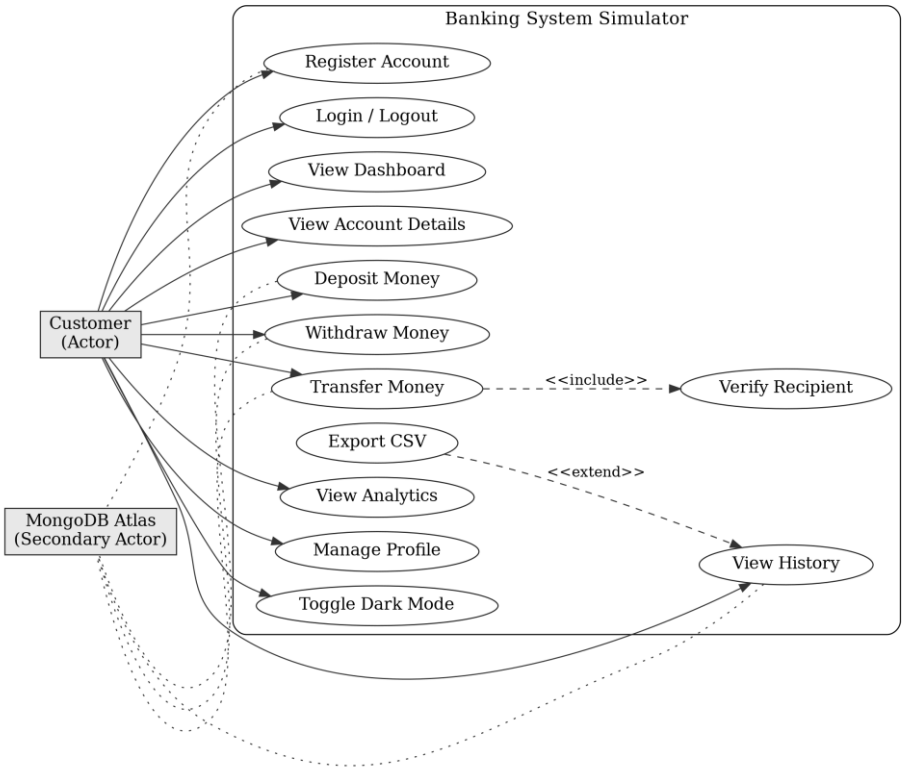


Figure 3.7: Use case diagram

3.6 Use Cases

The principal use cases are described in detail in Tables 3.1 to 3.4. Each table gives the name, actors, preconditions, the flow of events, the postconditions and the exceptions.

Table 3.1: Use case UC-01: Register Account

Element	Description
Name	Register Account
Actors	Customer; MongoDB
Preconditions	The customer is not logged in and the e-mail is not yet registered.
Flow of events	1. Customer opens the Register page and enters name, e-mail and password. 2. The client validates the fields and sends them to the server. 3. The server validates them again with Zod and checks the e-mail is free. 4. The password is hashed with bcrypt (cost 12) and the user is stored. 5. A unique 12-digit account number is generated and the account is created. 6. A deposit of Rs. 10,000 is recorded as the welcome credit. 7. A token is returned and the customer is taken to the dashboard.
Postconditions	User, account and welcome-credit transaction exist; the customer is logged in.
Exceptions	Duplicate e-mail (409); weak password or invalid name (400). If any step after user creation fails, the user and account are deleted.

Table 3.2: Use case UC-02: Deposit Money

Element	Description
Name	Deposit Money
Actors	Customer; MongoDB
Preconditions	The customer is logged in and the account is active.
Flow of events	1. Customer enters an amount or taps a quick amount. 2. The client shows a confirmation dialog with the balance after the deposit. 3. On confirm the client sends the amount with a new Idempotency-Key. 4. The server converts the amount to paise, atomically adds it to the balance and inserts a DEPOSIT record. 5. The receipt is returned and the balance on screen is refreshed.
Postconditions	Balance increased by the amount; one DEPOSIT record exists.
Exceptions	Invalid amount, more than two decimals or above Rs. 10,00,000 (400); repeated key (the earlier result is returned, no second deposit).

Table 3.3: Use case UC-03: Transfer Money

Element	Description
Name	Transfer Money
Actors	Customer; MongoDB
Preconditions	The customer is logged in; the recipient has an active account.
Flow of events	1. Customer enters the recipient’s 12-digit number and presses Verify. 2. The server returns the holder name and the masked number. 3. Customer enters the amount and confirms. 4. The server debits the sender, credits the recipient and writes two linked records inside one transaction (or with compensation). 5. The sent record is returned as the receipt.
Postconditions	Sender balance decreased and recipient balance increased by the same amount; both records share a transfer group.
Exceptions	Unknown or suspended recipient (404); own account (400); insufficient balance (409); any failure after the debit reverses all changes.

Table 3.4: Use case UC-04: View and Export Transaction History

Element	Description
Name	View and Export Transaction History
Actors	Customer; MongoDB
Preconditions	The customer is logged in.
Flow of events	1. Customer opens History; the first page of ten records is shown, newest first. 2. Customer enters search text, selects a type or date range or changes the sort order. 3. The server builds a filter restricted to the customer’s own user identifier and returns one page with the total. 4. Customer presses Export to receive the filtered or the full history as a CSV file.
Postconditions	The page shown matches the filters; an exported file contains only the customer’s records.

Element	Description
Exceptions	Invalid type or date (400); no matching record (empty state is shown).

CHAPTER 4: ALGORITHM DESIGN

This chapter is the algorithmic core of the report. After stating the invariants that the system must maintain, it presents eight algorithms in pseudocode, with flowcharts for the two most important, and analyses the time and space complexity and the correctness of each. Throughout the chapter n denotes the total number of transaction documents, m the number of documents that belong to one user and match a filter, k the page size and L the length of an input string.

4.1 Design Overview and Invariants

The correctness of the ledger rests on four invariants. They are stated first because every algorithm below is designed to preserve them, and Section 4.11 argues that it does.

- **I1 (non-negativity):** for every account, balance >= 0 at every instant.
- **I2 (conservation):** a transfer changes the sum of all balances by exactly zero; deposits add and withdrawals subtract exactly their amount.
- **I3 (exactly once):** for any pair (user, idempotency key), at most one ledger effect exists, however many copies of the request arrive.
- **I4 (ledger agreement):** for every account, the balance equals the sum of credits minus the sum of debits recorded in its transactions.

4.2 Algorithm 1: Exact Amount Parsing

Floating-point numbers cannot represent most decimal fractions exactly: the expression 0.1 + 0.2 evaluates to 0.30000000000000004 in JavaScript. Money handled this way drifts over time. The simulator therefore stores every amount as an integer number of **paise** (one hundredth of a rupee) and parses the user's decimal string with integer operations only, as in Algorithm 4.1.

Algorithm 4.1: toPaise(value)

```
INPUT : value, a string or number entered by the user
OUTPUT: paise, a positive integer


1  s <- trim(string(value))
2  if s does not match  ^[0-9]{1,9}(\.[0-9]{1,2})?$  then
3      raise Error("Enter a valid amount, at most 2 decimals")
4  split s at "." into rupees r and fraction f (f = "" if absent)
5  f <- pad f on the right with "0" to length 2
6  paise <- integer(r) * 100 + integer(f)
7  if paise <= 0 then raise Error("Amount must be positive")
8  if paise > 100,000,000 then raise Error("Maximum is Rs. 10,00,000")
9  return paise
```

The regular expression admits at most nine integer digits and at most two decimals, so exponent notation, negative values, empty strings and values such as “1e3” are rejected before any arithmetic. Every step is a constant-size operation on a string of bounded length, so the **time complexity is O(L)** with L at most 12, which is O(1), and the space is O(1). Because the result is an integer below 10^8, it is exactly representable in a double-precision number and all later additions and subtractions are exact.

4.3 Algorithm 2: Atomic Conditional Balance Update

This is the central algorithm. Instead of reading the balance, testing it in the program and writing it back, the program sends the database a single update request whose filter contains the condition. For a debit the filter is “this account, active, and balance at least the amount”, and the update is “add a negative delta”. The database engine evaluates the filter and applies the increment as one indivisible operation on the document. If no document matches, nothing was changed and the algorithm reports insufficient funds. The flowchart is given in Figure 4.1 and the pseudocode in Algorithm 4.2.

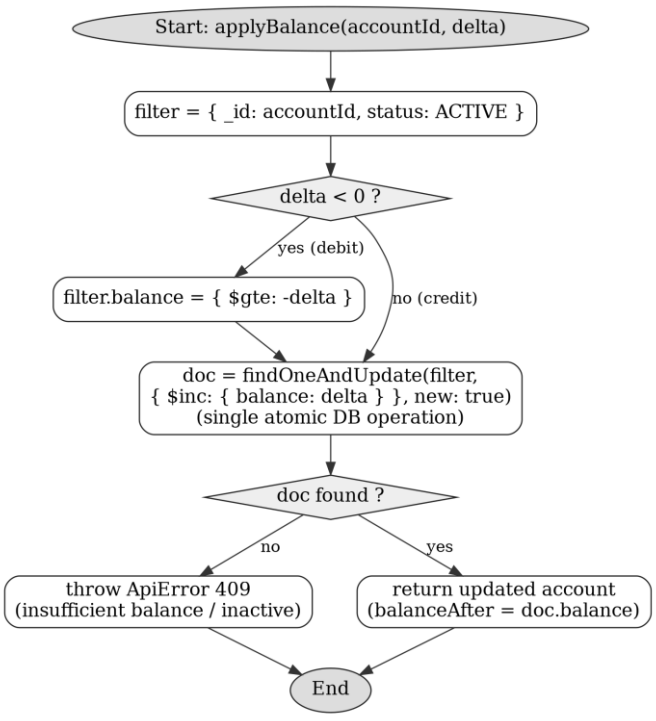


Figure 4.1: Flowchart of the atomic conditional balance update

Algorithm 4.2: applyBalance(accountId, delta)

```
INPUT : accountId, delta (positive = credit, negative = debit, in paise)
OUTPUT: the updated account document, or an error


1  filter <- { _id: accountId, status: "ACTIVE" }
2  if delta < 0 then filter.balance <- { $gte: -delta }
3  account <- findOneAndUpdate(filter, { $inc: { balance: delta } },
                               returnNew = true)      // one atomic step
4  if account = NULL then
5      raise Error(409, "Insufficient balance or account not active")
6  return account
```

Analysis. Line 3 uses the primary-key index on the account identifier, so it takes O(log N) time in the number N of accounts and O(1) extra space; with the identifier already known, the lookup in practice touches a handful of index pages. No loop and no retry are required, so the worst-case number of database round trips is one. **Correctness.** Suppose the balance is B and p concurrent debits of amount a arrive. Because the test and the decrement of each request are indivisible, the requests are serialized by the engine in some order. The j-th request in that order succeeds if and only if B - (j - 1)a >= a, so exactly min(p, floor(B / a)) requests succeed and the balance after all of them is B - a * min(p, floor(B / a)) >= 0. This is the formula that the experiment of Section 6.3 confirms: with B = 1000, a = 100 and p = 50, exactly ten succeed.

4.4 Algorithm 3: Idempotent Execution

Algorithm 4.3 wraps any money-moving work so that a repeated idempotency key has no second effect. It combines a fast pre-check with a safety net provided by the unique index.

Algorithm 4.3: idempotent(userId, key, work)

```
INPUT : userId, key (may be absent), work (a function that changes balances
        and inserts ledger records)
OUTPUT: the ledger record of the operation


1  if key is present then
2      prior <- find transaction with (userId, idempotencyKey = key)
3      if prior exists then return prior               // fast path
4  try
5      return runInTransactionOrFallback(work)
6  catch error e
7      if e is a duplicate-key error (code 11000) then
8          prior <- find transaction with (userId, idempotencyKey = key)
9          if prior exists then return prior           // lost the race
10     raise e
```

Analysis. The pre-check on lines 2 and 3 is a lookup in the unique index on (userId, idempotencyKey) and costs O(log n). Two copies of the same request that both pass the pre-check at the same moment both run the work, but only one can insert a ledger record carrying that key, because the unique index rejects the second insertion. The

second copy receives error 11000 on line 7 and returns the first copy's record. For the balance to be correct, the second copy's changes must be undone; with a database transaction the abort discards them, and without one the work function itself compensates, as described in Section 5.7. The invariant I3 therefore holds regardless of timing. The partial filter on the index applies it only to documents that actually have a key, so ordinary requests without a key are unaffected.

4.5 Algorithm 4: Transfer with Atomicity and Compensation

A transfer moves money between two different accounts and must keep invariant I2. Figure 4.2 shows the flowchart and Algorithm 4.4 the pseudocode. The sender is debited first, which is the step that can fail for business reasons (insufficient funds). If it fails, nothing has been changed. The credit to an active account practically cannot fail for business reasons, and the two ledger records are inserted last. If any step fails inside a database transaction, the transaction is aborted. If the database offers no transaction, the catch block applies the opposite increments to the accounts that were already changed.

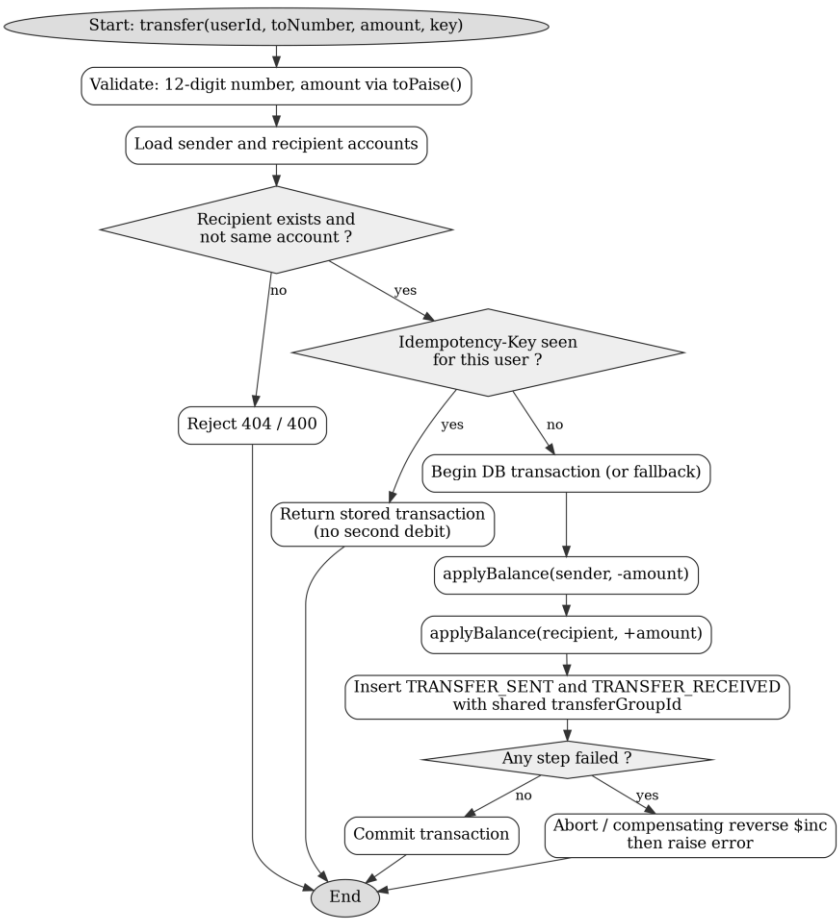


Figure 4.2: Flowchart of the transfer algorithm with idempotency and compensation

Algorithm 4.4: transfer(userId, toNumber, amount, key)

```
INPUT : userId of the sender, toNumber (12 digits), amount in paise, key
OUTPUT: the TRANSFER_SENT record, or an error


1  from <- account of userId;   to <- account with number toNumber
2  if from = NULL or to = NULL then raise Error(404)
3  if from.id = to.id then raise Error(400, "Cannot transfer to yourself")
4  group <- new random UUID
5  return idempotent(userId, key, function(session):
6      debited <- NULL;  credited <- NULL
7      try
8          debited  <- applyBalance(from.id, -amount, session)
9          credited <- applyBalance(to.id,   +amount, session)
10         sent <- insert TRANSFER_SENT     (debited,  amount, group, key)
11         insert TRANSFER_RECEIVED         (credited, amount, group)
12         return sent
13     catch error e
14         if session = NULL then            // no DB transaction available
15             if credited then increment to.balance   by -amount
16             if debited  then increment from.balance by +amount
17         raise e)
```

Analysis. The algorithm performs a constant number of indexed operations: two account lookups, two atomic updates and two inserts, so the time is O(log N + log n) and the space is O(1). **Correctness for I2.** The debit and credit have the same amount. In the failure-free case the sum of balances is unchanged. In every failure case, lines 14 to 16 (or the transaction abort) restore the pre-state, so the sum is again unchanged. **I1**

follows from Algorithm 4.2, since the debit uses the conditional update and the credit can only increase a balance. **I4** holds because the ledger records are inserted with the same balance snapshots that the two atomic updates returned. The experiment of Section 6.5 checks I2 on 300 concurrent transfers among ten accounts.

4.6 Algorithm 5: Unique Account Number Generation

Each account receives a 12-digit number made of the fixed prefix 40 and ten random digits. Randomness makes numbers unpredictable, which prevents a stranger from guessing a neighbouring number, but it can in principle collide with an existing number. Algorithm 4.5 is a **Las Vegas** randomized algorithm: it always returns a correct, unique number, and only its running time is random.

Algorithm 4.5: createAccountNumber()

```
OUTPUT: a 12-digit account number not used by any account


1  loop
2      n <- "40" + ten digits, each drawn uniformly from 0..9
3      if no account with accountNumber = n exists then return n
```

Analysis. There are 10^10 possible numbers. If N accounts already exist, the probability that one draw collides is p = N / 10^10, and the number of draws is geometrically distributed with expectation 1 / (1 - p). For one million accounts p = 10^-4 and the expected number of draws is 1.0001, so the **expected time is O(log N)** for the uniqueness lookup, with a negligible constant factor. The unique index on the account number is the final safety net: even if two registrations draw the same number simultaneously, only one insertion succeeds. The weakness of the scheme is that a failed insertion currently surfaces as an error; retrying the registration is an accepted way to handle that extremely rare case.

4.7 Algorithm 6: Filtered, Sorted and Paginated History

The history endpoint builds a filter from the query parameters, sorts by creation time and returns one page. The text search escapes all regular-expression metacharacters before building the pattern, so that a search for “(” or “.*” is treated as plain text and cannot cause expensive or malformed patterns. Algorithm 4.6 shows the procedure.

Algorithm 4.6: historyPage(userId, q)

```
INPUT : userId; q with optional type, from, to, text, sort, page, limit
OUTPUT: items (at most k records), total, number of pages


1  page <- max(1, int(q.page));  k <- min(50, max(1, int(q.limit or 10)))
2  filter <- { userId: userId }
3  if q.type present then require q.type in TYPES;  filter.type <- q.type
4  if q.from or q.to present then
5      parse as dates (reject invalid);  extend "to" to 23:59:59.999
6      filter.createdAt <- { $gte: from, $lte: to }
7  if q.text present then
8      p <- case-insensitive pattern of escape(first 60 chars of q.text)
9      filter.$or <- [ {description ~ p}, {transactionReference ~ p} ]
10 order <- createdAt ascending if q.sort = "oldest" else descending
11 items <- find(filter).sort(order).skip((page-1)*k).limit(k)
12 total <- count(filter)
13 return items, total, max(1, ceil(total / k))
```

Analysis. With the compound index (userId, createdAt) the database positions itself at the first entry of the user in O(log n) and reads entries in the requested order, so no separate sort is needed. Skipping (page - 1) * k entries costs O(page * k), and reading the page costs O(k). The count on line 12 is O(m) in the worst case, where m is the number of the user's documents that match. The total time is therefore **O(log n + page * k + m)** and the space is O(k) for the returned page. When the text search is present, the regular expression is evaluated against each of the user's documents, which is O(m * L), but even then the collection is first narrowed to the user through the index, so the cost does not depend on the other customers' data. Section 6.8 measures the effect.

The CSV export reuses the same filter with a cap of 10,000 rows, and prefixes any cell that begins with =, +, -, @, tab or carriage return with an apostrophe so that a spreadsheet treats it as text and not as a formula.

4.8 Algorithm 7: Analytics Aggregation

The analytics service answers three questions: totals by type, a time series per month or year, and the distribution of types. Each is a group-by over the completed transactions of one user within an optional date range. Algorithm 4.7 shows the monthly series.

Algorithm 4.7: monthlySeries(userId, from, to, format)

```
INPUT : userId, optional date range, format "%Y-%m" or "%Y"
OUTPUT: list of (period, deposits, withdrawals, income, expense, count)

1  S <- documents with userId, status = COMPLETED, createdAt in [from, to]
2  H <- empty hash table  (key = period string)
3  for each document d in S do
4      key <- format(d.createdAt)
5      add d.amount to H[key].deposits      if d.type = DEPOSIT
6      add d.amount to H[key].withdrawals   if d.type = WITHDRAWAL
7      add d.amount to H[key].income        if d.type in {DEPOSIT,
TRANSFER_RECEIVED}
8      add d.amount to H[key].expense       if d.type in {WITHDRAWAL,
TRANSFER_SENT}
9      H[key].count <- H[key].count + 1
10 return the entries of H sorted by key
```

Analysis. Selecting S uses the compound index and costs O(log n + m). The loop makes one hash-table update per document, which is O(1) expected, so the loop is O(m). The final sort is O(g log g) for g groups, and g is at most the number of months in the range, which is tiny compared with m. The overall time is **O(log n + m + g log g)** and the space is O(g). The definition of income and expense guarantees that a transfer is counted once for each user: the payer's document is TRANSFER_SENT (expense) and the payee's is TRANSFER_RECEIVED (income), and each user queries only their own documents.

4.9 Algorithm 8: Authentication and Rate Limiting

Authentication consists of hashing and verifying passwords with bcrypt and issuing and verifying signed tokens. Rate limiting protects the login and registration routes from guessing attacks. Algorithm 4.8 summarises the three parts.

Algorithm 4.8: Authentication and rate limiting

```
register(name, email, password):
1  validate(name, email, password)            // length, letter, digit
2  h <- bcrypt(password, cost = 12, random salt)    // time ~ 2^cost
3  store user (name, email, h);  create account;  record welcome credit
4  return sign(HMAC-SHA256, { sub: user.id, exp: now + 1 day })

requireAuth(request):
5  token <- text after "Bearer " in the Authorization header
6  payload <- verify signature and expiry of token     // O(1)
7  user <- find user by payload.sub                    // O(log U)
8  if payload invalid or user = NULL then reject 401
9  attach user to request

rateLimit(request):                              // fixed window
10 c <- counter for (client address, current 15-minute window)
11 c <- c + 1
12 if c > 20 then reject 429 "Too many attempts"
```

Analysis. Hashing and verification cost Theta(2^cost) operations by design; the experiment of Section 6.10 shows 333 ms at cost 12. This cost is paid once per login or

registration and is the main defence against offline guessing if the database were stolen. Token verification is an HMAC computation over a short string plus one indexed lookup, so a request is authenticated in O(log U) for U users. The fixed-window counter needs O(1) time per request and O(A) space for A active client addresses. Its known weakness is that a client can send a burst at the end of one window and another at the start of the next; for the small limit used here this is acceptable.

4.10 Complexity Summary

Table 4.1 collects the results of the preceding sections.

Table 4.1: Time and space complexity of the algorithms

Algorithm	Time	Space	Technique
1. toPaise	O(L), L <= 12	O(1)	Integer arithmetic, regular expression
2. applyBalance	O(log N)	O(1)	Atomic compare-and-set update
3. idempotent	O(log n) plus work	O(1)	Unique index, check then insert
4. transfer	O(log N + log n)	O(1)	Atomic updates, transaction or compensation
5. createAccountNumber	Expected O(log N)	O(1)	Las Vegas randomization
6. historyPage	O(log n + page*k + m)	O(k)	Compound index, skip and limit
7. monthlySeries	O(log n + m + g log g)	O(g)	Hash aggregation, sorting
8. authentication	Theta(2^cost); O(log U) per request	O(1) per request	bcrypt, HMAC token, fixed window

4.11 Correctness Arguments

I1 (non-negativity). Every decrease of a balance in the system goes through Algorithm 4.2 with a negative delta, and that algorithm applies the decrease only when the balance is at least the amount. Therefore a balance can only be reduced to a value that is zero or greater. As a second line of defence, the Mongoose schema declares a minimum of zero and requires an integer. No other code path writes to the balance field.

I2 (conservation). Deposits and withdrawals change one balance by exactly the recorded amount. A transfer changes two balances by +a and -a. If the transfer fails after the first update, the transaction abort or the compensation in Algorithm 4.4 applies

the opposite change, so that the net effect is zero. The compensation uses the same integer amount, so no rounding is possible.

I3 (exactly once). Of two concurrent copies of a request with the same key, only one ledger record carrying the key can be inserted, because the unique index rejects the second. The other copy receives a duplicate-key error, its changes are undone, and it returns the stored record, so exactly one set of effects remains.

I4 (ledger agreement). Each atomic update returns the new balance, and the ledger record is created with balanceAfter equal to that value. Records are never modified, so by induction the balance equals the signed sum of the records. A process killed between the balance update and the record insertion on a standalone server is the one case that is not covered; Chapter 7 lists a reconciliation job as future work.

CHAPTER 5: IMPLEMENTATION

This chapter describes how the design of Chapters 3 and 4 was turned into a working application. It lists the technologies, the structure of the code, the main parts of the backend and frontend, the programming interface, the security measures, the testing strategy and the problems that were met during development.

5.1 Technology Stack

The project is written entirely in JavaScript. The tools and libraries are listed in Table 5.1. All of them are free and open source, and the database can be hosted at no cost on MongoDB Atlas.

Table 5.1: Tools and libraries used

Layer	Technology	Purpose
Frontend	React 18, Vite	Component-based single-page application and fast development server
Frontend	Tailwind CSS 4	Utility-first styling, responsive layout and dark mode
Frontend	React Router, Recharts, Lucide, React Hot Toast	Routing and route guards, charts, icons, notifications
Backend	Node.js, Express 4	REST API and middleware pipeline
Backend	Mongoose 8	Schemas, validation and indexes for MongoDB
Backend	Zod	Server-side validation of request bodies
Backend	bcryptjs, jsonwebtoken	Password hashing and signed tokens
Backend	Helmet, CORS, express-rate-limit	Secure headers, origin control, request throttling
Database	MongoDB (Atlas)	Document storage, atomic updates, transactions on replica sets
Testing	node:test, mongodb-memory-server	15 integration tests against a real MongoDB instance
Tools	Visual Studio Code, Git, Chrome	Development, version control and demonstration

5.2 Backend Implementation

5.2.1 The ledger service

The ledger service, in the file ledger.js, implements Algorithms 4.2 to 4.5. The function that applies a balance change is short, and its essential lines are shown below. The filter contains the balance condition for debits, and the update is a single increment.

```
async function applyBalance(accountId, delta, session) {
  const filter = { _id: accountId, status: "ACTIVE" };
  if (delta < 0) filter.balance = { $gte: -delta };
  const acc = await Account.findOneAndUpdate(filter,
      { $inc: { balance: delta } }, { new: true, ...opts(session) });
  if (!acc) throw new ApiError(409, delta < 0
      ? "Insufficient balance or account is not active"
      : "Account is not available");
  return acc;
}
```

The function that runs a unit of work inside a MongoDB transaction first tries to open one. On a standalone server it receives an error that says that transactions need a replica set, and it then runs the same work without a session. In that case the compensating code of Algorithm 4.4 provides atomicity. This design lets the same code run unchanged on a laptop and on Atlas.

5.3 REST Interface

Table 5.2 lists the endpoints. All routes except registration, login and the health check require a valid bearer token, and all money-moving routes accept an optional Idempotency-Key header that must be eight to 64 letters, digits, underscores or hyphens.

Table 5.2: REST API endpoints

Method and path	Purpose	Auth
GET /api/health	Server health check	No
POST /api/auth/register	Create user, account and welcome credit	No (rate limited)
POST /api/auth/login	Verify credentials, return token	No (rate limited)
GET /api/auth/me, POST /api/auth/logout	Current user; logout acknowledgement	Yes
POST /api/auth/change-password	Change password after verifying the old one	Yes
GET /api/accounts/me	Own account details and balance	Yes
GET /api/accounts/verify/:number	Resolve a 12-digit number to holder name and masked number	Yes
GET /api/accounts/:id	Account by identifier (owner only; otherwise 403)	Yes
POST /api/transactions/deposit	Deposit	Yes
POST /api/transactions/withdraw	Withdraw	Yes
POST /api/transactions/transfer	Transfer to another account	Yes
GET /api/transactions	History page with filters, sort and pagination	Yes
GET /api/transactions/export	CSV export of filtered or all records	Yes
GET /api/transactions/:id	One transaction (owner only)	Yes

Method and path	Purpose	Auth
GET /api/analytics/summary, /monthly, /distribution	Totals, time series, type distribution	Yes
GET and PATCH /api/users/me	Read profile; change name only	Yes

5.4 Frontend Implementation

The client uses an authentication context, a small API module and one component per page, with a route guard for protected pages. Money forms follow one pattern. The amount is validated in the browser for immediate feedback, a confirmation dialog shows the balance that will result, and the request is sent with an idempotency key generated when the dialog opens. The key is reused if the user presses the button again before the response returns, which makes accidental double submission harmless. After a successful response, a toast notification is shown and the balance is refreshed from the server rather than computed locally, so the screen always shows the stored value.

Figures 5.1 to 5.4 show the main screens of the running application.

[Insert screenshot: Dashboard]

Figure 5.1: Dashboard with balance, statistics and charts

[Insert screenshot: Transfer page after pressing Verify]

Figure 5.2: Transfer page with recipient verification

[Insert screenshot: History page]

Figure 5.3: Transaction history with search, filters and pagination

[Insert screenshot: Analytics page]

Figure 5.4: Analytics page with charts

5.5 Security Measures

Security follows defence in depth. Passwords are stored only as bcrypt hashes (cost 12) with a random salt. Every protected route verifies a one-day JWT, and account and transaction reads check ownership, returning 403 or 404 for another customer's data. Zod validates every body on the server, and a strict schema stops mass assignment of role or e-mail. The exact amount parser bounds amounts, the history search escapes regular-expression characters, and CSV cells are sanitized against formula injection. The login and register routes allow 20 requests per 15 minutes, Helmet sets secure headers, CORS is limited to the client origin, request bodies are limited to 10 KB, login errors do not reveal whether an e-mail exists, and account numbers are masked in lists.

5.6 Testing Strategy

The test suite contains 15 integration tests written with the built-in Node.js test runner. Each test starts the real Express application against a real MongoDB instance and sends HTTP requests, so the tests exercise the routes, validation, authentication, the ledger and the database indexes together. The tests were run twice: against a MongoDB replica set, where transfers use transactions, and against a standalone server, where the compensation path is used. The list of tests and the results are in Section 6.2.

5.7 Challenges Faced and Solutions

Table 5.3: Challenges and their solutions

Challenge	Solution
Floating-point rounding of money	Store integer paise; parse with a regular expression and integer arithmetic (Algorithm 4.1)
Concurrent withdrawals overdrawing an account	Atomic conditional update instead of read-check-write (Algorithm 4.2); verified by experiment
Retries creating duplicate transfers	Idempotency key with a unique partial index and a pre-check (Algorithm 4.3)
Transfers failing half-way on a standalone MongoDB	Try a transaction first; fall back to compensating updates (Algorithm 4.4)

Challenge	Solution
Double credit on standalone MongoDB when duplicate requests race (found during the evaluation)	The duplicate request had already added to the balance when its ledger insert failed. The single-entry path now reverses its own balance change when no transaction is available. After the fix 20 of 20 trials ended with the correct balance, against 0 of 20 before it.
Configuration mistakes during first run (placeholder secret, wrong variable name, Atlas network rules and password)	Fail-fast startup validation, clear error messages and a documented checklist
Frontend could not reach the API (CORS and port mismatch)	Client origin read from CLIENT_URL; the client API address from VITE_API_URL; restart after changing environment files
Large bundle warning from the charting library	Accepted for this scope; code splitting is listed as future work

The double-credit defect in the fifth row deserves emphasis because it shows the value of measuring. The ledger insert was protected by the unique index, so the number of transaction records was always correct (one per key). The functional test for duplicate submissions used the transfer route, which already compensated for failures, and the single-entry deposit and withdrawal route was not covered by a test of concurrent duplicates. Only an experiment that compared the final balance with the expected balance under concurrent duplicate deposits revealed that the balance had been increased more than once. The defect could not occur on a replica set, where the transaction abort discards the extra increment, which is the configuration of MongoDB Atlas.

5.8 Combined Experiment: Queue, Stack and Searching

To connect the simulator with the basic data structures of the course, a small standalone program, bank_ds_demo.py, combines three of them in one banking scenario. It was written in Python and its chart was drawn with the seaborn library. A **queue** holds pending transactions and releases them in first-in first-out order, so the oldest request is always served first. A **stack** holds the recent history of every account, so the latest record is on top, can be read first and can be popped to undo it. **Searching** finds the account that a transaction refers to by its account number. In the web application the same ideas appear in other forms: the history page shows the newest record first, and recipient verification is a search by account number answered by an index, as described in Section 2.6.

Table 5.4: Data structures used in the combined experiment

Structure	Role in the experiment	Operations	Cost
Queue (deque)	Pending transactions in arrival order	enqueue, dequeue	O(1) each
Stack (list)	Recent history of one account, latest on top	push, pop, peek top k	O(1) push and pop
Linear search	Baseline account lookup	scan every account	O(n)
Binary search	Account lookup in a list sorted by number	halve the range	O(log n)
Hash table (dict)	Lookup by account number	key access	O(1) expected

The core of the program is shown below. The queue is a deque, the stack is a list of which only the end is used, and the processing loop dequeues one transaction, finds its account with binary search, applies it and pushes a record on the stack of that account.

```
class Account:
    def __init__(self, number, holder, balance=0):
        self.number, self.holder, self.balance = number, holder, balance
        self.history = []                       # STACK: append = push, pop() = pop

    def push(self, record):  self.history.append(record)
    def pop(self):           return self.history.pop() if self.history else None
    def recent(self, k=3):   return list(reversed(self.history[-k:]))

class TransactionQueue:                         # QUEUE (FIFO) using deque
    def   init  (self): self. q = deque()
    def enqueue(self, tx): self._q.append(tx)
    def dequeue(self):     return self. q.popleft()
    def __len__(self):     return len(self._q)

def linear search(accounts, number):            # O(n)
    steps = 0
    for acc in accounts:
        steps += 1
        if acc.number == number: return acc, steps
    return None, steps

def binary search(accounts, number):            # O(log n), list sorted by number
    lo, hi, steps = 0, len(accounts) - 1, 0
    while lo <= hi:
        steps += 1
        mid = (lo + hi) // 2
        if accounts[mid].number == number: return accounts[mid], steps
        if accounts[mid].number < number: lo = mid + 1
        else: hi = mid - 1
    return None, steps

def process queue(queue, accounts, log, totals):
    while len(queue):
        tx = queue.dequeue()                    # FIFO: oldest request first
        acc, _ = binary_search(accounts, tx["acc"])        # SEARCH
        if acc is None:
            log.append(f"REJECT  #{tx['id']:02d}  account {tx['acc']} not found")
            continue
        if tx["type"] == "Withdrawal" and acc.balance < tx["amt"]:
            log.append(f"REJECT  #{tx['id']:02d}  {acc.holder}: insufficient
balance "
                       f"for Rs.{tx['amt']}")
            continue
        acc.balance += tx["amt"] if tx["type"] == "Deposit" else -tx["amt"]
        acc.push((tx["id"], tx["type"], tx["amt"]))      # STACK push
        totals.append({"Account": acc.holder, "Type": tx["type"],
```

```
                        "Amount": tx["amt"]})
        log.append(f"OK      #{tx['id']:02d}  {tx['type']:<10} Rs.{tx['amt']:>6}  "
                   f"{acc.holder:<7} balance Rs.{acc.balance}")
```

The program queued 25 transactions for six accounts, each starting with Rs. 5,000, including one for an account number that does not exist. Part of the output is shown below. The queue processed the requests in the order of their numbers, the unknown account was rejected without disturbing the others, and the stack of each account listed its latest records first. Popping the top of the stack of Aman removed record 18 and exposed record 17, which is how a last action is undone.

```
QUEUE: processing 25 pending transactions in FIFO order
OK      #01  Deposit    Rs.  2500  Chetan  balance Rs.7500
OK      #02  Deposit    Rs.   500  Farah   balance Rs.5500
OK      #03  Deposit    Rs.  1500  Eshan   balance Rs.6500
OK      #04  Deposit    Rs.  4000  Eshan   balance Rs.10500
...
OK      #24  Withdrawal Rs.  6000  Divya   balance Rs.1000
REJECT  #25  account 409999999999 not found

STACK: most recent 3 records per account (latest first)
  Aman    [(18, 'Deposit', 2500), (17, 'Deposit', 4000), (9, 'Deposit', 6000)]
  Chetan  [(21, 'Deposit', 1000), (15, 'Deposit', 500), (1, 'Deposit', 2500)]

SEARCH: find account by number
  linear : Eshan in 5 steps
  binary : Eshan in 2 steps
```

The binary search needs a list sorted by account number and takes about log2 n steps, whereas the linear search takes n/2 steps on average. Table 5.5 shows the measured effect for 100,000 accounts and 1,000 random lookups.

Table 5.5: Account search on 100,000 accounts (average of 1,000 lookups)

Method	Average steps	Average time (ms)
Linear search	48,729	2.08361
Binary search	15.7	0.00836
Hash table (dict)	1	0.00059

The linear search needed about 48700 comparisons per lookup against about 16 for the binary search, and the hash table needed one. In time this was 2.08 ms, 0.0084 ms and 0.0006 ms, so binary search was about 249 times faster than linear search and the hash table was about 14 times faster again. This agrees with the O(n), O(log n) and O(1) growth rates. The price of binary search is that the list must be kept sorted, and the price of the hash table is extra memory. A database index, as used by the simulator, gives the logarithmic behaviour of binary search without keeping a sorted list in the application.

Finally, the accepted transactions were summed by account and type, and the totals were drawn as the grouped bar chart of Figure 5.5, using the seaborn library.

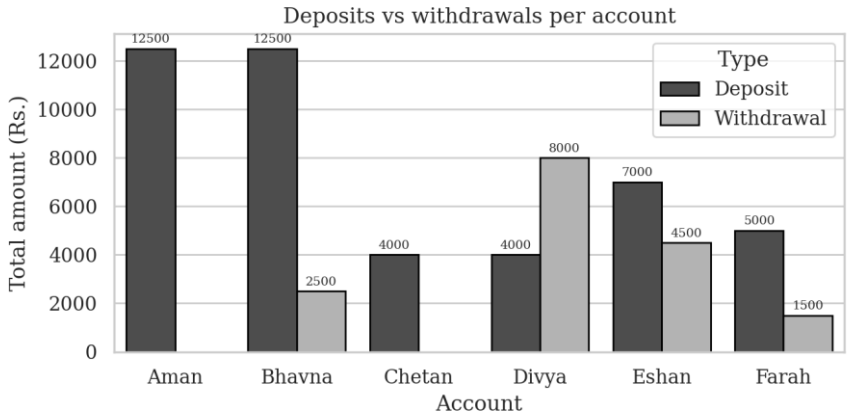


Figure 5.5: Deposits vs withdrawals per account (seaborn bar chart)

The chart shows at a glance that Aman and Bhavna received the most deposits (Rs. 12,500 each), while Divya withdrew more (Rs. 8,000) than she deposited (Rs. 4,000). It is the same comparison that the analytics page of the web application draws with Recharts for the logged-in customer.

CHAPTER 6: RESULTS AND DISCUSSIONS

This chapter reports the outcome of the automated tests and of eight experiments that were designed to check the invariants and the complexity claims of Chapter 4. All numbers in the tables were produced by running the scripts on the project's own code; none was estimated.

6.1 Experimental Setup

The experiments ran on a single Linux machine, with the Node.js application and MongoDB 8.3 on the same host, so that network latency is negligible and the numbers show the cost of the algorithms and the database rather than of the network. Two database configurations were used. The **standalone** configuration is a single mongod process without a replica set, which exercises the compensation path. The **replica-set** configuration is a single-node replica set, which enables multi-document transactions and exercises the transactional path. The experiments call the ledger service directly, without the HTTP layer and without bcrypt, except where stated, so that the measurement isolates the algorithm under study. The eight experiments are described in Sections 6.3 to 6.10.

6.2 Functional Test Results

All 15 integration tests passed on both the replica-set and the standalone configuration. Their coverage is summarised below.

The tests cover registration with password hashing, duplicate e-mail and weak password rejection, login, blocked unauthorized access, deposit and withdrawal with invalid and insufficient amounts, transfer with invalid recipient and self-transfer, idempotency under concurrent duplicates, concurrent withdrawals that never overdraw, concurrent transfers that keep the total constant, denial of cross-user access, history filtering and pagination, CSV scoping, analytics without double counting, and profile editing limited to the name.

6.3 Experiment 1: Concurrent Withdrawals

Method. An account was given a balance of Rs. 1,000. Fifty withdrawals of Rs. 100 were sent at the same time. Two implementations were compared: a naive one that reads the balance, checks it in the program and writes back the reduced value, and the atomic

conditional update of Algorithm 4.2. By the formula of Section 4.3, at most ten withdrawals may succeed. The test was repeated in 20 independent trials for each implementation.

Table 6.1: Results of Experiment 1 (20 trials of 50 concurrent withdrawals)

Measure	Naive read-modify-write	Atomic conditional update
Withdrawals accepted per trial (mean)	50.0 of 50	10.0 of 50
Maximum accepted in any trial	50	10
Trials in which more was paid out than the balance	20 of 20	0 of 20
Trials in which balance disagrees with the accepted withdrawals	20 of 20	0 of 20

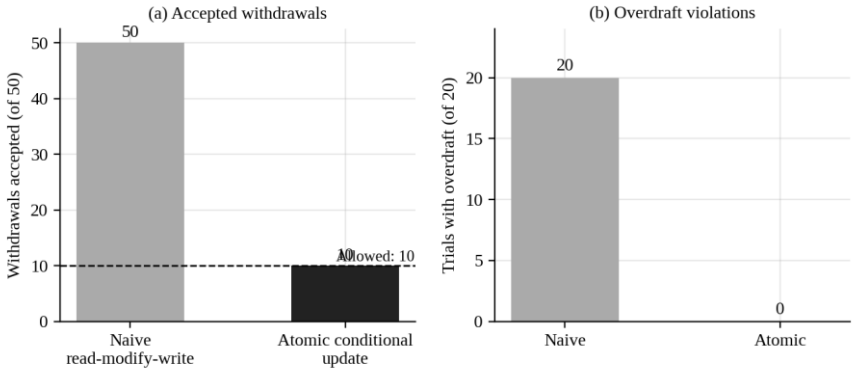


Figure 6.1: Accepted withdrawals and overdraft violations, naive vs atomic

Discussion. The naive algorithm accepted all fifty withdrawals in every trial, paying out Rs. 5,000 from an account that held Rs. 1,000, and its recorded balance disagreed with the money paid out in all 20 trials. The atomic algorithm accepted exactly ten in every trial and never overdrew. The result matches the closed form min(p, floor(B / a)) = min(50, 10) = 10 derived in Section 4.3, and it was identical on the standalone and the replica-set configuration because the guarantee comes from the single atomic operation and not from the transaction.

6.4 Experiment 2: Idempotency under Duplicate Requests

Method. Ten copies of a deposit of Rs. 500 carrying the same idempotency key were sent concurrently, in 20 trials. After each trial, the number of ledger records for the user and the final balance were checked. Correct behaviour is one record and a balance of exactly Rs. 500.

Table 6.2: Results of Experiment 2 (20 trials, 10 concurrent duplicates each)

Configuration	Ledger records equal to one	Balance exactly Rs. 500
Replica set (transactions)	Yes, in all 20 trials	20 of 20
Standalone, before the fix	Yes, in all 20 trials	0 of 20
Standalone, after the fix	Yes, in all 20 trials	20 of 20

Discussion. In every configuration the unique index kept the number of ledger records at one, so the idempotency record itself was correct. On the standalone server before the fix, however, the balance was wrong in every trial, because the losing copies had already added the amount before their insertion failed. The transactional configuration was correct because the abort removed the extra increments. After the single-entry path was changed to reverse its own balance change when there is no transaction, the standalone configuration also reached 20 of 20. This experiment is the reason for the fifth row of Table 5.3.

6.5 Experiment 3: Conservation under Concurrent Transfers

Method. Ten accounts, each with Rs. 1,000, performed 300 concurrent transfers of between Rs. 10 and Rs. 30 in a rotating pattern, so that every account was both sender and receiver at the same time. The sum of all balances was compared before and after, and the accounts were checked for negative balances.

Table 6.3: Results of Experiment 3

Measure	Standalone	Replica set
Transfers attempted	300	300
Total money before (paise)	1,000,000	1,000,000
Total money after (paise)	1,000,000	1,000,000
Money conserved (invariant I2)	Yes	Yes
Accounts with a negative balance (invariant I1)	0	0

Discussion. The total was unchanged and no balance became negative in either configuration, which supports the arguments of Section 4.11 for I1 and I2 under concurrent load.

6.6 Experiment 4: Operation Latency

Method. Each operation was executed 200 times sequentially, and the time of each call was recorded. The mean and the 95th percentile are reported in Table 6.4. The times

are for the ledger service and the database only; the HTTP layer, token verification and bcrypt are excluded.

Table 6.4: Results of Experiment 4: latency in milliseconds (200 sequential calls each)

Operation	Standalone mean	Standalone p95	Replica-set mean	Replica-set p95
Deposit	5.27	11.23	5.72	10.49
Withdrawal	4.83	9.89	5.50	7.14
Transfer	6.69	13.30	8.49	11.25

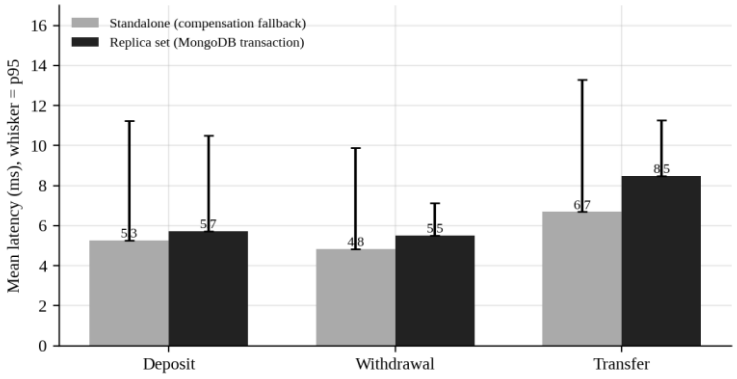


Figure 6.2: Mean latency of ledger operations with 95th percentile whiskers

Discussion. A deposit or withdrawal takes about five milliseconds and a transfer about seven to eight, which is consistent with the constant number of indexed operations in the analysis of Algorithms 4.2 and 4.4: a transfer performs two updates and two inserts and therefore costs roughly 1.3 to 1.5 times a single-entry operation. The transactional configuration adds a small overhead (about 0.5 to 0.7 ms for single-entry operations and 1.8 ms for transfers on average), which is the price of the stronger atomicity guarantee.

6.7 Experiment 5: Throughput under Contention

Method. One thousand deposits were issued at once, spread over 20 accounts, so that about 50 requests competed for each account at any moment. The elapsed time was measured, and throughput is the number of operations divided by the time.

Discussion. The standalone configuration completed about 252 operations per second and the replica set about 13 under this extreme level of contention. We attribute the large difference to write conflicts between concurrent transactions on the same account document, which the driver resolves by aborting and retrying, but this explanation was not measured directly. The result is a genuine trade-off between strict atomicity and

throughput for hot accounts. The experiment is deliberately harsh: an interactive banking demo or a typical retail account sees a handful of operations per account per second, a load for which the sequential results of Section 6.6 apply. If hot accounts mattered, the algorithm could use the atomic conditional update without a transaction for single-entry operations, as the standalone path does, and keep transactions for transfers.

6.8 Experiment 6: History Retrieval and the Index

Method. The transaction collection was grown to 1,000, 5,000, 20,000, 50,000 and 100,000 documents, with one tenth of them belonging to the test user. At each size the first page (ten records, newest first) was fetched, once with the compound index on (userId, createdAt) and once from an identical collection that had no such index. Each figure is the mean of repeated runs after a warm-up call.

Table 6.5: Results of Experiment 6: latency of the first history page (ms)

Documents in collection	User's documents	Without index	With index	Speed-up
1,000	100	2.29	1.19	1.9x
5,000	500	5.74	1.25	4.6x
20,000	2,000	24.17	0.79	30.8x
50,000	5,000	34.79	0.75	46.2x
100,000	10,000	99.27	1.06	94.1x

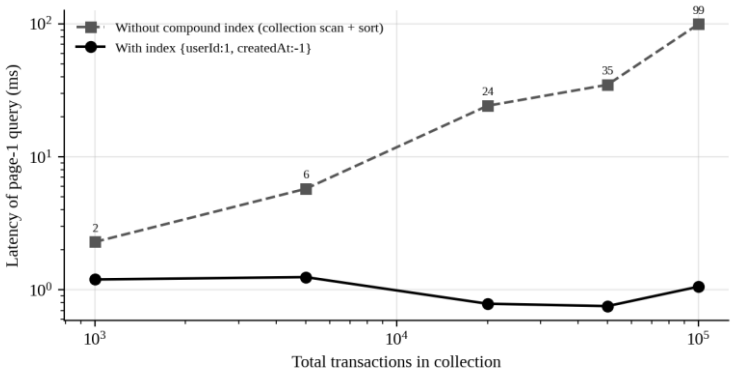


Figure 6.3: History page latency with and without the compound index

The cost of deep pages was measured on the 100,000-document collection, where the user owns 10,000 documents.

Table 6.6: Results of Experiment 6 (continued): latency by page number with the index (ms)

Page requested	Rows skipped	Latency
1	0	0.89
10	90	1.50
100	990	1.92
500	4,990	7.81
900	8,990	12.34

Discussion. Without the index the latency grew roughly in proportion to the collection size, from 2.3 ms at 1,000 documents to 99.3 ms at 100,000, which is the O(n) scan predicted in Section 2.6. With the index the latency stayed at about one millisecond at every size, because the database positions itself at the user's first entry and reads ten entries in order, so the cost does not depend on n. At 100,000 documents the index gave a speed-up of about 94 times. The page-number experiment confirms the O(page * k) term of Algorithm 4.6: the latency rose from under a millisecond on the first page to about 12 ms on page 900. Even the deepest page is fast enough for interactive use, so offset pagination is acceptable for this application, while keyset pagination would remove the growth if accounts become much larger.

6.9 Experiment 7: Analytics Aggregation

Method. The monthly aggregation of Algorithm 4.7 was run for the user who owns 10,000 documents in a collection of 102,620 documents, and the mean of ten runs was taken.

Result and discussion. The aggregation took 200.0 ms on the standalone configuration and 195.5 ms on the replica set, which is about 20 microseconds per document. This is consistent with the O(m) term of the analysis, since every one of the user's documents is visited once. The time is independent of the more than 90,000 other documents, because the index restricts the scan to the user. Most real accounts have far fewer than 10,000 records, and an aggregation of that size still completes in a fifth of a second.

6.10 Experiment 8: Cost of Password Hashing

Method. A password was hashed and verified with the bcryptjs library at cost factors 8, 10, 12 and 14, and the time of one hash was recorded.

Table 6.7: Results of Experiment 8: bcrypt time per operation (ms)

Cost factor	Iterations (2^cost)	Hash time	Verify time
8	256	60	20
10	1,024	86	79
12	4,096	333	340
14	16,384	1475	1457

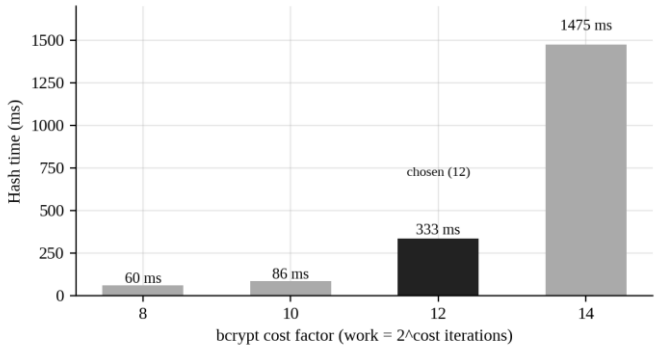


Figure 6.4: bcrypt hashing time against the cost factor

Discussion. The time grows exponentially with the cost factor, as designed. The chosen cost of 12 needs about a third of a second per operation, which a legitimate user does not notice at login but which limits an attacker with a stolen hash database to a few guesses per second per core. Cost 14 would take almost one and a half seconds with this pure JavaScript implementation, which would make login feel slow and would let a flood of login requests occupy the server, a reason why the rate limiter of Algorithm 4.8 is important.

6.11 Discussion

Taken together, the experiments support the central claims of the project. The atomic conditional update of Algorithm 4.2 is both simple and necessary: the naive alternative failed in every trial, while the atomic version matched the closed-form prediction exactly. The idempotency design of Algorithm 4.3 kept the ledger correct under duplicate requests, and the evaluation uncovered and repaired a defect that the functional tests had not exposed. The conservation experiment showed that the compensation design preserves the total amount of money under concurrent transfers. The index experiments confirmed the complexity analysis: retrieval cost depends on the page requested and not on the size of the collection.

The results also show limits. The transactional configuration is slower for single operations and much slower for contended hot accounts, offset pagination slows on very deep pages, and the compensation approach does not protect against a process killed between two steps. The experiments ran on one machine with a single-node replica set, so absolute numbers on a managed cloud cluster will be higher, although relative behaviour such as growth with collection size should be the same.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project designed, analysed, implemented and evaluated a banking system simulator whose core is a small set of algorithms that keep a ledger correct under concurrency, retries and partial failure. The system was documented with a context diagram, data flow diagrams at three levels, an ER diagram, a use case diagram and four detailed use cases. Eight algorithms were presented in pseudocode with complexity analysis: exact amount parsing, atomic conditional balance update, idempotent execution, transfer with compensation, randomized account number generation, indexed and paginated history retrieval, hash aggregation for analytics, and authentication with rate limiting. Four invariants (no overdraft, conservation of money, exactly-once effect and agreement between balance and ledger) were stated and argued for.

The implementation, in React, Node.js, Express and MongoDB, passes all 15 automated integration tests on a transactional and on a non-transactional database. The atomic conditional update accepted exactly the ten withdrawals that a balance of Rs. 1,000 allows, while the naive version accepted all fifty in every trial; 300 concurrent transfers conserved the total money; and a compound index kept the history page at about one millisecond for up to 100,000 documents, where the unindexed query rose to about 99 ms. A small combined experiment with a queue, a stack and account search showed binary search to be hundreds of times faster than linear search on 100,000 accounts. The evaluation also uncovered and repaired a real defect in the non-transactional path, which shows that measurement can reveal faults that functional tests do not.

The main lesson of the work is that correctness in a banking system is a property of algorithms and not of screens. The same user interface could sit on a naive or on a correct ledger, and only analysis and experiment tell the two apart.

7.2 Future Work

- **Administrator dashboard.** Use the existing role field to build screens for user management, suspension of accounts and monitoring of total money in the system.

- **Reconciliation job.** A background process that recomputes each balance from the ledger and reports any difference would verify invariant I4 continuously and repair the rare crash case on standalone servers.
- **Keyset pagination.** Replace skip and limit by a cursor on (createdAt, id), which reduces the cost of any page to O(log n + k).
- **Token revocation.** Short-lived access tokens with refresh tokens and a revocation list would allow real logout and forced sign-out.
- **Sliding-window or token-bucket rate limiting** with a shared store, which removes the burst weakness of the fixed window and works across several server instances.
- **Multiple accounts and account types,** interest calculation as a scheduled batch algorithm, and a statement generator.

REFERENCES

[1] J. Gray and A. Reuter, Transaction Processing: Concepts and Techniques. San Francisco, CA, USA: Morgan Kaufmann, 1993.

[2] T. Haerder and A. Reuter, “Principles of transaction-oriented database recovery,” ACM Computing Surveys, vol. 15, no. 4, pp. 287-317, 1983.

[3] P. A. Bernstein, V. Hadzilacos, and N. Goodman, Concurrency Control and Recovery in Database Systems. Reading, MA, USA: Addison-Wesley, 1987.

[4] N. Provos and D. Mazieres, “A future-adaptable password scheme,” in Proc. USENIX Annual Technical Conference, FREENIX Track, 1999, pp. 81-91.

[5] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[7] MongoDB, Inc., “Transactions,” MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/

[8] Stripe, Inc., “Idempotent requests,” Stripe API Reference. [Online]. Available: https://stripe.com/docs/api/idempotent_requests

[9] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O’Reilly Media, 2017.

[10] OWASP Foundation, “Password Storage Cheat Sheet” and “CSV Injection.” [Online]. Available: https://owasp.org

[11] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.

[12] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[13] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O’Neil, and P. O’Neil, “A critique of ANSI SQL isolation levels,” in Proc. ACM SIGMOD Int. Conf. Management of Data, 1995, pp. 1-10.

[14] Meta Platforms, Inc., “React documentation.” [Online]. Available: https://react.dev